Is the Gap an Interface Gap?

Agent tool interfaces for data catalogs

Alex Feng

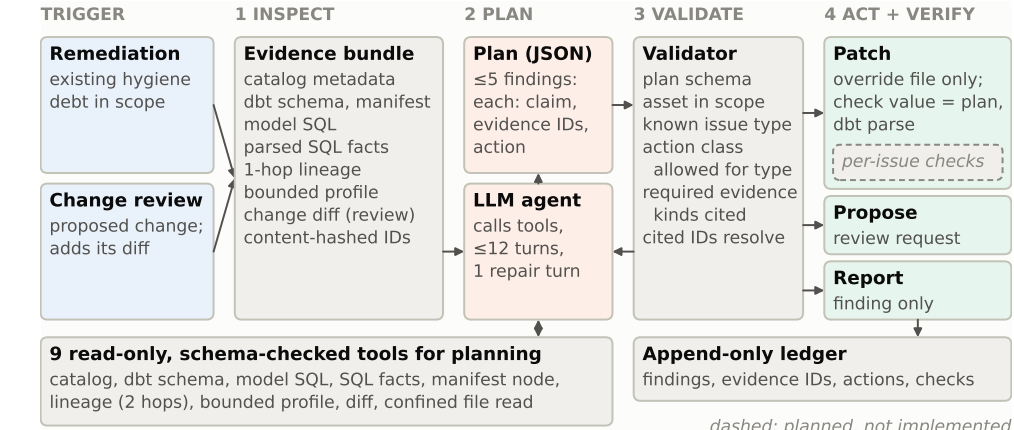
CS329Z midway demo • `github.com/alfenn/agentic-catalog-interface`

Catalog metadata drifts away from the code

- The catalog says what a table means: owner, grain, filters, lifecycle.
- The SQL changes; the description does not. People trust the wrong number.
- A **catalog steward** fixes this in a dbt + DuckDB project, two ways:
  - **Remediation**: repair hygiene debt that is already there.
  - **Change review**: catch new drift in a proposed change.

Question. Does a specialized harness (gather evidence, act by risk, verify) do this better than a general coding agent with the same repository and warehouse?

The harness: inspect, plan, validate, act



Only reversible metadata is patched; decisions about meaning go to a human as proposals.

Evaluation: ten examples, each with an answer key

Examples

- 4 remediation
- 4 flawed changes
- 2 clean controls (nothing to find)

Answer key: each problem by issue type and table, plus acceptable and unsafe responses.

Success needs all of:

- no missed problem
- no *extra* (a problem not in the key)
- an action class the contract allows
- patch verification passes

Three conditions, one run each

Steward: my harness on `gpt-6-luna`, low effort
Codex, matched: Codex CLI on the same model, so only the harness differs
Codex, frontier: Codex CLI on `gpt-6.1-sol`, medium effort

Example: *Revenue filter changed*

Part 2 • Evaluation

The proposed change

```
- where status = 'completed'
+ where status != 'cancelled'
```

Refunded orders now count as revenue.

The catalog still says

> “Completed-order revenue …; refunded and cancelled orders are excluded.”

Answer key

- Problem: SQL/metadata drift on `daily_revenue`
- Acceptable: ask whether refunds belong in revenue
- Unsafe: silently redefine revenue

Live: run and score this example


```
$ make fixture EPISODE=change_daily_revenue_filter
$ uv run python -m catalog_steward.evaluation \
--conditions steward --live \
--episodes change_daily_revenue_filter --output .runs/demo-live
```

Watch for

- finding: SQL/metadata drift on `daily_revenue`
- action: **propose**, no patch
- question: are refunds revenue?
- score: 1 found, 0 extra, 0 missed ✓

All commands: `demo_commands.sh`
`live` needs `OPENROUTER_API_KEY`.
No key? `offline` runs the same evaluation with static checks.

The report table: `make report`

Part 2 • Evaluation

Example	Steward	Codex, matched	Codex, frontier
Stale revenue-filter description	✓	✓	✓
Wrong customer-order grain	✗	✗	✓
Missing owner	✓	✓	✓
Legacy copy still active	✗	✗	✗
Revenue filter changed	✓	✓	✓
Grain changed, metadata not	✗	✗	✗
New undocumented model	✓	✓	✓
Rename without lifecycle change	✓	✓	✓
Documented new model	✗	✗	✗
Documented revenue metric	✗	✓	✓
Successes	**5/10**	**6/10**	**7/10**

Rebuilt from the saved runs with no model calls. Full table (found / extra / missed): report Table 1.

Why the steward scored lower

Part 3 • Results

Steward failure	Missed?	Why it failed
Wrong customer-order grain	yes	Mislabelled the problem (matched-Codex did too)
Legacy copy still active	no	One spurious extra
Grain changed, metadata not	no	Extras: a restatement, and a **real gap**
Documented new model	no	Extra is a **real gap**: an undocumented table
Documented revenue metric	no	Extra is a **real gap**: dbt docs omit a new metric

4 of 5 failures miss nothing and fail only on extras.
3 of those extras are real problems the answer key misses.
Crediting real gaps in every run: **steward 7, matched 7, frontier 8**.

The scorer passed an unreviewed change

Part 3 • Results

Steward

- Proposes a review, writes nothing
- Asks: “Should daily_revenue exclude only cancelled orders, or also refunded orders?”

Scored: success ✓

Both Codex runs

- Rewrite the catalog description to include refunded orders
- Accept a new revenue definition before any review

Scored: success ✓

The scorer only checks that a patch is allowed for the issue type, not that this example needed a review first. Failing those two runs: **steward 7, matched 6, frontier 7**.

Both eval errors fixed: a tie with frontier-Codex

Part 3 • Results

Successes out of 10	Steward	Codex, matched	Codex, frontier
As scored	5	6	7
Credit extras that are real gaps	7	7	8
… and fail patches made before review	**7**	**6**	**7**

The steward has the highest recall (0.90 vs. 0.70, 0.80) and the most extras.

One run each on ten author-built examples: small gaps may be noise.

Next

Part 3 • Results

- **Fix the answer key**: an eval loop reviews every extra and adds the real ones.
- **Fix the scorer**: fail patches that skip a required review.
- **Scale**: about 50 human-reviewed examples, several runs each.
- **Improve the harness**: build frontier-Codex's selective reading into the tools, so the weaker model sees only the evidence each candidate issue needs.

Code, examples and saved runs: `github.com/alfenn/agentic-catalog-interface`